INFORME: RECORRIDO E INSERCIÓN EN UNA LISTA ENLAZADA

Caso de trabajo: gestión dinámica de solicitudes de atención

3. Inserción al final

El algoritmo recorre los nodos hasta encontrar el último y enlaza allí el nuevo caso. Si la lista está vacía, el nuevo nodo pasa a ser el primero. En el ejemplo, se agrega “Caso 40”.

Inserción al final

```
def insertar_final(inicio, dato):
    nuevo = Nodo(dato)
    if inicio is None:
        return nuevo
    actual = inicio
    while actual.siguiente is not None:
        actual = actual.siguiente
    actual.siguiente = nuevo
    return inicio

inicio = insertar_final(inicio, "Caso 40")
```

4. Inserción después de un nodo dado

Se busca el nodo de referencia. Al encontrarlo, el nuevo nodo apunta al siguiente y luego el nodo de referencia apunta al nuevo. Así se inserta “Caso 25” después de “Caso 20” sin perder el resto de la lista. Si no existe la referencia, la lista queda sin cambios.

Inserción después de un nodo

```
def insertar_despues(inicio, referencia, dato):
    actual = inicio
    while actual is not None:
        if actual.dato == referencia:
            nuevo = Nodo(dato)
            nuevo.siguiente = actual.siguiente
            actual.siguiente = nuevo
            return inicio
        actual = actual.siguiente
    print("Referencia no encontrada")
    return inicio

inicio = insertar_despues(inicio, "Caso 20", "Caso 25")
```

Explicación final: ¿cómo cambia la estructura?

Operación	Cambio en la lista
Recorrido	Se visitan los nodos; no se modifica la estructura.
Inserción al inicio	El nuevo nodo pasa a ser el primero y apunta al antiguo inicio.
Inserción al final	El último nodo apunta al nuevo caso.
Inserción después de un nodo	El nuevo caso se ubica entre el nodo de referencia y el que le seguía.

Conclusión: una lista enlazada permite registrar, agregar y reubicar solicitudes modificando las referencias entre nodos, sin desplazar todos los registros. Esto facilita gestionar una secuencia que cambia constantemente.